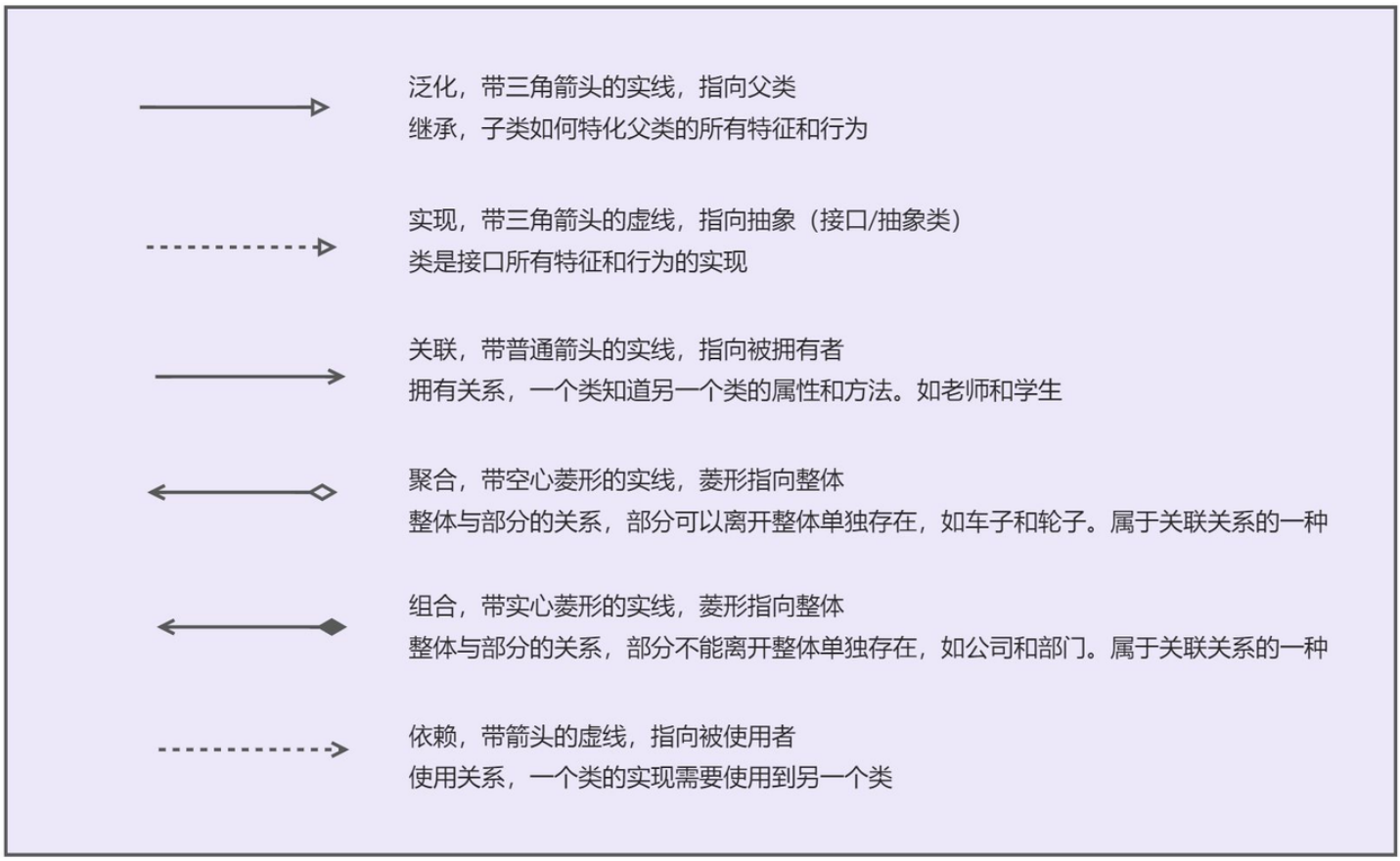
dtr实现的再分析

类图表示



各种关系的强弱顺序：泛化=实现>组合>聚合>关联>依赖

体现类与类、类与接口之间的**纵向**关系

- **泛化/继承**（generalization）
- **实现**（realization）

体现类与类、类与接口之间的**横向**关系

- **依赖**（dependence）：只要是在类中用到了对方（成员变量、局部变量、方法的形参、方法的返回类型），那么他们之间就存在依赖关系。
- 依赖关系的特例：
  - **关联**（association）：A类依赖于B对象，并且把B作为A的一个成员变量，则A和B存在关联关系。两个类是一个层次的，不存在部分跟整体之间的关系。
  - 关联关系的特例：

- **聚合**（aggregtion）：在聚合关系中，两个类是处在不平等层次上的，一个代表整体，另一个代表部分，整体和部分可以分开。
- **组合**（composition）：这种关系比聚合更强，也称为强聚合。整体和个体不能独立存在，一定是在一个模块中同时管理整体和个体，生命周期必须相同。

参考：

- UML一一 类图关系 (泛化、实现、依赖、关联、聚合、组合)
- 一文弄懂UML类图（六大关系：依赖，泛化，实现，关联，聚合，组合）

别名

```cpp
template<typename T>
using Ref = intrusive_ptr<RefCell<T>>;

using strong = intrusive_ptr<CheckpointTensorCell>;
using strongs = std::vector<strong>;
using weak = weak_intrusive_ptr<CheckpointTensorCell>;
using weaks = std::vector<weak>;

using Tensors = std::vector<Tensor>;
using rematerialize_function_t = std::function<Tensors(const Tensors&)>;
using mutate_function_t = std::function<void(const Tensors&)>;

using time_t = std::chrono::time_point<std::chrono::system_clock>;
using duration_t = std::chrono::system_clock::duration;

using ecn_ptr = intrusive_ptr<EquivalentClassNode<CheckpointInfo>>;
```

CheckpointInfo 结构体

```cpp
struct CheckpointInfo {
  duration_t compute_cost;
  /**
   * 被以下函数调用:
   * - double AliasPool::cost
   */
  double cost(size_t memory, size_t staleness) const {
    TORCH_CHECK(memory > 0);
    TORCH_CHECK(staleness > 0);
    return compute_cost.count() / static_cast<double>(memory * staleness);
  }
```

```
    CheckpointInfo(duration_t compute_cost) :
      compute_cost(compute_cost) {
    }
};
```

CheckpointPool 结构体

```
struct CheckpointPool {
  std::vector<weak_intrusive_ptr<AliasPool>> aps;
  std::vector<weak_intrusive_ptr<External>> exts;
  std::random_device rd;
  std::mt19937 gen = std::mt19937(rd());
  bool sample_tensors = true;
  bool ignore_small_tensors = true;
  bool has_memory_budget = false;
  long memory_budget;
  void evict();
  void auto_evict();
  void add(const intrusive_ptr<AliasPool>&);
  CheckpointPool();
};

/**
 * 被以下函数调用:
 * - void AliasPool::set_not_evicted
 * - MakeRawResult make_raw
*/
void CheckpointPool::add(const intrusive_ptr<AliasPool>& p) {
  if (p->memory > 0 && (memory_count == 0 || !ignore_small_tensors || p->memory >= 0.01 * double(memory_sum/memory_count))) {
    aps.push_back(weak_intrusive_ptr<AliasPool>(p));
  }
}

/**
 * 被以下函数调用:
 * - void Rematerializer::remat
 * - MakeRawResult make_raw
*/
void CheckpointPool::auto_evict() {
  if (has_memory_budget) {
    while (current_memory() > memory_budget) {
      evict();
    }
```

```
37      }
38    }
39
40    /**
41     * 被以下函数调用:
42     * - void CheckpointPool::auto_evict
43    */
44    long current_memory() {
45      auto device_stat = c10::cuda::CUDACachingAllocator::getDeviceStats(0);
46      return device_stat.allocated_bytes[0].current;
47    }
48
49    /**
50     * 被以下函数调用:
51     * - void CheckpointPool::auto_evict
52    */
53    void CheckpointPool::evict() {
54      time_t pre = std::chrono::system_clock::now();
55      TORCH_CHECK(aps.size() > 0);
56      bool shrunk = false;
57      int evict_idx = -1;
58      double evict_cost = INFINITY;
59      time_t current_time = std::chrono::system_clock::now();
60      auto remove_from_aps = [&](size_t i) {
61                               aps[i] = aps[aps.size() - 1];
62                               aps.pop_back();
63                             };
64      std::uniform_int_distribution<> distrib(1, 1 * std::max(1, static_cast<int>
    (std::sqrt(aps.size()))));
65      for (size_t i = 0; i < aps.size();) {
66        auto cannot_evict = [&]() {
67                              shrunk = true;
68                              remove_from_aps(i);
69                            };
70        auto ap_strong = aps[i].lock();
71        if (!ap_strong.defined()) {
72          cannot_evict();
73        }
74        else if (ap_strong->ecn) {
75          cannot_evict();
76        }
77        else {
78          if (ap_strong->evictable()) {
79            double cost = ap_strong->cost(current_time);
80            if (cost < evict_cost) {
81              evict_cost = cost;
82              evict_idx = i;
```

```
          }
        }

        if (sample_tensors) {
          i += distrib(gen);
        } else {
          i += 1;
        }
      }
    }
    if (evict_idx == -1) {
      TORCH_CHECK(shrunk);
    } else {
      auto evict_from_idx = [&](size_t idx) {
                              auto ap_strong = aps[idx].lock();
                              TORCH_CHECK(ap_strong.defined());
                              ap_strong->evict();
                              remove_from_aps(evict_idx);
                            };
      evict_from_idx(evict_idx);
    }
    time_t post = std::chrono::system_clock::now();
    search_time_ += (post - pre).count();
  }

  /**
   * 被以下函数调用:
   * - void CheckpointPool::evict
   */
  double AliasPool::cost(time_t current_time) {
    time_t pre = std::chrono::system_clock::now();
    auto cpi = head_remat->get_cpi();
    auto ecns = neighbor_ecn();
    for (const auto& necn : ecns) {
      cpi = merge_cpi(cpi, get_t(necn));
    }
    auto ret = cpi.cost(memory, (current_time - last_used_time).count());
    time_t post = std::chrono::system_clock::now();
    cost_time_ += (post - pre).count();
    return ret;
  }

  /**
   * 被以下函数调用:
   * - double AliasPool::cost
   */
  CheckpointInfo Rematerializer::get_cpi() {
```

```
130     return CheckpointInfo(ecn ? duration_t(0) : compute_cost);
131   }
132
133   /**
134    * 被以下函数调用:
135    * - double AliasPool::cost
136    * - void AliasPool::evict
137   */
138   CheckpointInfo merge_cpi(CheckpointInfo l, CheckpointInfo r) {
139     return CheckpointInfo(l.compute_cost + r.compute_cost);
140   }
141
142   /**
143    * 被以下函数调用:
144    * - void CheckpointPool::evict
145   */
146   void AliasPool::evict() {
147     TORCH_CHECK(!ecn);
148     ecn = head_remat->get_ecn();
149     auto ecns = neighbor_ecn();
150     for (const auto& necn : ecns) {
151       merge<CheckpointInfo>(merge_cpi, ecn, necn);
152     }
153     TORCH_CHECK(memory > 0);
154     TORCH_CHECK(lock_count == 0);
155     TORCH_CHECK(!is_evicted);
156     is_evicted = true;
157     for (const weak& w : tensors) {
158       if (auto cell = w.lock()) {
159         cell->evict();
160       }
161     }
162   }
163
164   /**
165    * 被以下函数调用:
166    * - void AliasPool::evict
167   */
168   ecn_ptr Rematerializer::get_ecn() {
169     if (!ecn) {
170       ecn = ecn_ptr::make(CheckpointInfo(compute_cost));
171     }
172     return ecn;
173   }
174
```

External 结构体

```
struct External : intrusive_ptr_target {
  External(const strong& value) : value(value) {
    value->pool->register_external();
  }
  External(const Tensor& value) :
    External(strong::make(value,
                          intrusive_ptr<AliasPool>::make(Unsafe(),

intrusive_ptr<Rematerializer>(),
                                                         memory(value)))) { }
  External(const Tensor& value,
           const intrusive_ptr<AliasPool>& pool,
           const intrusive_ptr<Rematerializer>& remat) :
    External(strong::make(value, pool, remat)) { }
  strong value;
  void release_resources() override;
};
```

Rematerializer 结构体

```
struct Unsafe { };

struct Rematerializer : intrusive_ptr_target {
  rematerialize_function_t func;
  strongs inputs;
  weaks outputs;
  duration_t compute_cost;
  ecn_ptr ecn;
  Rematerializer(const Unsafe&,
                 const rematerialize_function_t& func,
                 const strongs& inputs,
                 duration_t compute_cost)  :
    func(func),
    inputs(inputs),
    compute_cost(compute_cost) {
  }
  void release_resources() final {
    func = rematerialize_function_t();
    inputs.clear();
    outputs.clear();
  }
```

```
22    void remat();
23    ecn_ptr get_ecn();
24    CheckpointInfo get_cpi();
25  };
26
27  /**
28   * 被以下函数调用:
29   * - Tensor CheckpointTensorCell::get
30  */
31  void Rematerializer::remat() {
32    for (const strong& s : inputs) {
33      s->pool->lock();
34    }
35    Tensors ts = uncheckpoint(inputs);
36    time_t pre = std::chrono::system_clock::now();
37    auto ret = func(ts);
38    time_t post = std::chrono::system_clock::now();
39    pool.auto_evict();
40    remat_compute_time_ += (post - pre).count();
41    TORCH_CHECK(ret.size() == outputs.size());
42    for (size_t i = 0; i < outputs.size(); ++i) {
43      if (auto output_cell = outputs[i].lock()) {
44        output_cell->fill(ret[i]);
45      }
46    }
47    ecn.reset();
48    for (const strong& s : inputs) {
49      s->pool->unlock();
50    }
51  }
52
53  /**
54   * 被以下函数调用:
55   * - void Rematerializer::remat
56   * - MakeRawResult make_raw
57  */
58  Tensors uncheckpoint(const strongs& inputs) {
59    Tensors ret;
60    ret.reserve(inputs.size());
61    for (const strong& input : inputs) {
62      ret.push_back(input->get());
63    }
64    return ret;
65  };
66
67  /**
68   * 被以下函数调用:
```

```
 * - void Rematerializer::remat
*/
void CheckpointTensorCell::fill(const Tensor& t) {
  if (!(this->t)) {
    this->t = std::make_unique<Tensor>(t.detach());
    pool->set_not_evicted(pool);
    if (!defined) {
      defined = true;
      is_undefined_tensor = !t.defined();
      key_set_ = t.key_set();
      if (t.requires_grad()) {
        key_set_ = key_set_.add(DispatchKey::Autograd);
      }
      dtype_ = t.dtype();
      optional_device_ = t.optional_device();
    }
  }
}

/**
 * 被以下函数调用:
 * - void CheckpointTensorCell::fill
*/
void AliasPool::set_not_evicted(const intrusive_ptr<AliasPool>& self) {
  if (unlikely(is_evicted)) {
    is_evicted = false;
    if (ecn) {
      TORCH_CHECK(head_remat);
      auto cpi = get_t(ecn);
      update_t(ecn, CheckpointInfo(cpi.compute_cost - head_remat->compute_cost));
      ecn.reset();
    }
    pool.add(self);
  }
}
```

AliasPool 结构体

```
struct AliasPool : intrusive_ptr_target {
  weaks tensors;
  weaks neighbors;
  std::set<ecn_ptr> neighbor_ecn();
  size_t lock_count = 0;
```

```
  size_t external_count = 0;
  void lock() {
    ++lock_count;
  }
  void unlock() {
    --lock_count;
  }
  intrusive_ptr<Rematerializer> head_remat;
  bool evictable() const {
    return lock_count == 0 && head_remat;
  }
  bool is_evicted = false;
  size_t memory;
  time_t last_used_time;
  AliasPool(const Unsafe&, intrusive_ptr<Rematerializer> head_remat, size_t memory) :
    head_remat(head_remat),
    memory(memory),
    last_used_time(std::chrono::system_clock::now()) {
  }
  ecn_ptr ecn;
  double cost(time_t current_time);
  void evict();
  void register_external() {
    ++external_count;
  }
  void release_external() {
    --external_count;
    if (external_count == 0) {
      if (lock_count > 0) {return;}
      TORCH_CHECK(lock_count == 0);
      if (memory > 0 && (!ecn) && head_remat) {
        evict();
      }
    }
  }
  void set_not_evicted(const intrusive_ptr<AliasPool>& self);
  void release_resources() final {
    tensors.clear();
    neighbors.clear();
    head_remat.reset();
  }
};
```

CheckpointTensorCell 结构体

```
struct CheckpointTensorCell : intrusive_ptr_target {
  std::unique_ptr<Tensor> t;
  bool defined = false;
  bool is_undefined_tensor;
  DispatchKeySet key_set_;
  DispatchKeySet key_set() const {
    TORCH_CHECK(defined);
    return key_set_;
  }
  caffe2::TypeMeta dtype_;
  caffe2::TypeMeta dtype() const {
    TORCH_CHECK(defined);
    return dtype_;
  }
  c10::optional<Device> optional_device_;
  c10::optional<Device> optional_device() const {
    TORCH_CHECK(defined);
    return optional_device_;
  }
  intrusive_ptr<AliasPool> pool;
  intrusive_ptr<Rematerializer> remat;
  /**
   * 被以下函数调用:
   * - void AliasPool::evict
  */
  void evict() {
    TORCH_CHECK(remat);
    t.reset();
  }
  void fill(const Tensor& t);
  explicit CheckpointTensorCell(const Tensor& t, const
intrusive_ptr<AliasPool>& pool) : pool(pool) {
    fill(t);
  }
  explicit CheckpointTensorCell(const Tensor& t,
                                const intrusive_ptr<AliasPool>& pool,
                                const intrusive_ptr<Rematerializer>& remat) :
    pool(pool), remat(remat) {
    fill(t);
  }
  size_t memory() {
    TORCH_CHECK(defined);
    return pool->memory;
  }
  /**
   * 被以下函数调用:
```

```
     * - void pin
     * - int64_t dim
     * - int64_t numel
     * - IntArrayRef sizes
     * - int64_t size
     * - IntArrayRef strides
     * - int64_t stride
     * - Tensor uncheckpoint
     * - Tensor decheckpoint
     * - Tensors uncheckpoint
    */
    Tensor get() {
      if (! t) {
        TORCH_CHECK(remat);
        remat->remat();
      }
      TORCH_CHECK(t);
      TORCH_CHECK(! t->key_set().has(DispatchKey::CheckpointTensorId));
      pool->last_used_time = std::chrono::system_clock::now();
      return *t;
    }
    /**
     * 被以下函数调用:
     * - void pin
     * - void clear_checkpointpool
    */
    void pin() {
      get();
      pool->head_remat.reset();
      remat.reset();
    }
    void release_resources() final {
      t.reset();
      pool.reset();
      remat.reset();
    }
  };
```

CheckpointTensorImpl 结构体

```
  struct CheckpointTensorImpl : TensorImpl {
    int id = gen_counter();
    static int counter;
    static int gen_counter() {
```

```
    return counter++;
  }
  std::string counter_name() const {
    return std::string("x") + std::to_string(id);
  }

  Ref<intrusive_ptr<External>> ref;

  void release_resources() final;

  explicit CheckpointTensorImpl(const Ref<intrusive_ptr<External>>& ref) :
    TensorImpl(convert_key_set(ref->value->value->key_set()),
               ref->value->value->dtype(),
               ref->value->value->optional_device()),
    ref(ref) {
    if (key_set().has(DispatchKey::Autograd)) {
      set_requires_grad(true);
    }
  }

  explicit CheckpointTensorImpl(const intrusive_ptr<External>& e) :
    CheckpointTensorImpl(Ref<intrusive_ptr<External>>::make(e)) { }

  explicit CheckpointTensorImpl(const Tensor& t);

  static Tensors make(const std::string& name,
                      const rematerialize_function_t& remat,
                      const Tensors& inputs);

  // mutate_idx indicate which of the inputs will get mutated.
  static void mutate(const std::string& name,
                     const mutate_function_t& mutate,
                     const Tensors& inputs,
                     const std::vector<size_t>& mutate_idx);
  intrusive_ptr<TensorImpl> shallow_copy_and_detach(const VariableVersion&
version_counter,
                                                    bool
allow_tensor_metadata_change) const override;
  void shallow_copy_from(const c10::intrusive_ptr<TensorImpl>& impl) override;
  int64_t dim() const override {
    return ref->value->value->get().dim();
  }
  int64_t numel() const override {
    return ref->value->value->get().numel();
  }
  IntArrayRef sizes() const override {
    return ref->value->value->get().sizes();
```

```
50    }
51    int64_t size(int64_t d) const override {
52      return ref->value->value->get().size(d);
53    }
54    IntArrayRef strides() const override {
55      return ref->value->value->get().strides();
56    }
57    int64_t stride(int64_t d) const override {
58      return ref->value->value->get().stride(d);
59    }
60    bool has_storage() const override {
61      return false;
62    }
63  };
64
65  /**
66   * 被 aten/src/ATen/native/Checkpoint.cpp 中的函数调用
67  */
68  Tensors CheckpointTensorImpl::make(const std::string& name,
69                                     const rematerialize_function_t& remat,
70                                     const Tensors& inputs) {
71    Tensors checkpointed_inputs = try_checkpoint(inputs);
72    auto input_size = checkpointed_inputs.size();
73
74    strongs input_values;
75    input_values.reserve(input_size);
76
77    std::vector<std::string> args;
78    args.reserve(input_size);
79
80    for (const Tensor& t: checkpointed_inputs) {
81      auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
82      TORCH_CHECK(cpti);
83      input_values.push_back(cpti->ref->value->value);
84      if (use_log_) {
85        args.push_back(cpti->counter_name());
86      }
87    }
88
89    auto ret = make_raw(remat, input_values);
90
91    Tensors tensors;
92    tensors.reserve(ret.outputs.size());
93
94    for (const auto& t: ret.outputs) {
95      auto cp = Tensor(intrusive_ptr<CheckpointTensorImpl>::make(t));
96      tensors.push_back(cp);
```

```
97      }
98
99      return tensors;
100   }
101
102   /**
103    * 被以下函数调用:
104    * - Tensors CheckpointTensorImpl::make
105    * - void CheckpointTensorImpl::mutate
106   */
107   Tensors try_checkpoint(const Tensors& inputs) {
108     Tensors ret;
109     ret.reserve(inputs.size());
110     for (const Tensor& input : inputs) {
111       ret.push_back(at::native::try_checkpoint(input));
112     }
113     return ret;
114   }
115
116   /**
117    * 被以下函数调用:
118    * - Tensors try_checkpoint
119   */
120   Tensor try_checkpoint(const Tensor& t) {
121     return is_checkpoint(t) ? t : checkpoint(t);
122   }
123
124   /**
125    * 被以下函数调用:
126    * - Tensor try_checkpoint
127   */
128   bool is_checkpoint(const Tensor& t) {
129     auto* cpti = dynamic_cast<CheckpointTensorImpl*>(t.unsafeGetTensorImpl());
130     return cpti != nullptr;
131   }
132
133   /**
134    * 被以下函数调用:
135    * - Tensor try_checkpoint
136   */
137   Tensor checkpoint(const Tensor& t) {
138     auto cpti = intrusive_ptr<CheckpointTensorImpl>::make(t);
139     return Tensor(cpti);
140   }
141
142   /**
143    * 被以下函数调用:
```

```
144    * - Tensors CheckpointTensorImpl::make
145    * - void CheckpointTensorImpl::mutate
146   */
147   MakeRawResult make_raw(const rematerialize_function_t& remat_f,
148                          const strongs& inputs) {
149     for (const strong& s : inputs) {
150       s->pool->lock();
151     }
152     Tensors raw_inputs = uncheckpoint(inputs);
153     time_t pre = std::chrono::system_clock::now();
154     auto raw_outputs = remat_f(raw_inputs);
155     time_t post = std::chrono::system_clock::now();
156     pool.auto_evict();
157     base_compute_time_ += (post - pre).count();
158     std::vector<intrusive_ptr<External>> outputs;
159     std::vector<int> aliases;
160     weaks weak_outputs;
161     auto remat = intrusive_ptr<Rematerializer>::make(Unsafe(), remat_f, inputs,
    post - pre);
162
163     for (const Tensor& t : raw_outputs) {
164       intrusive_ptr<AliasPool> alias_pool;
165       int alias = get_alias(raw_inputs, t);
166       if (alias == -1) {
167         auto m = memory(t);
168         alias_pool = intrusive_ptr<AliasPool>::make(Unsafe(), remat, m);
169         pool.add(alias_pool);
170       }
171       else {
172         alias_pool = inputs[alias]->pool;
173         if (alias_pool->head_remat) {
174           alias_pool->head_remat->compute_cost += (post - pre);
175         }
176       }
177       auto e = intrusive_ptr<External>::make(t, alias_pool, remat);
178       pool.exts.push_back(weak_intrusive_ptr<External>(e));
179       alias_pool->tensors.push_back(weak(e->value));
180       outputs.push_back(e);
181       aliases.push_back(alias);
182       weak_outputs.push_back(weak(outputs.back()->value));
183     }
184     remat->outputs = weak_outputs;
185     for (size_t i = 0; i < inputs.size(); ++i) {
186       for (size_t j = 0; j < outputs.size(); ++j) {
187         if (!is_alias(raw_inputs[i], raw_outputs[j])) {
188           add_neighbor(inputs[i], outputs[j]->value);
189         }
```

```
    }
  }
  for (const strong& s : inputs) {
    s->pool->unlock();
  }
  return {outputs, aliases, post - pre, remat};
}

struct MakeRawResult {
  std::vector<intrusive_ptr<External>> outputs;
  std::vector<int> aliases;
  duration_t time;
  intrusive_ptr<Rematerializer> rematerializer;
};

size_t memory_sum = 0;
size_t memory_max = 0;
size_t memory_count = 0;

/**
 * 被以下函数调用:
 * - MakeRawResult make_raw
 */
inline size_t memory(const Tensor& t) {
  if (! t.has_storage()) {
    return 0;
  }
  auto& storage = t.storage();
  size_t res = storage.nbytes();
  memory_sum += res;
  memory_max = std::max(memory_max, res);
  memory_count += 1;
  return res;
}
```

compute_cost 出现的地方

```
struct CheckpointInfo {
  duration_t compute_cost;
  double cost(size_t memory, size_t staleness) const {
    TORCH_CHECK(memory > 0);
    TORCH_CHECK(staleness > 0);
    return compute_cost.count() / static_cast<double>(memory * staleness);
  }
```

```
  CheckpointInfo(duration_t compute_cost) :
    compute_cost(compute_cost) {
  }
};

CheckpointInfo merge_cpi(CheckpointInfo l, CheckpointInfo r) {
  return CheckpointInfo(l.compute_cost + r.compute_cost);
}
```

```
struct Rematerializer : intrusive_ptr_target {
  rematerialize_function_t func;
  strongs inputs;
  weaks outputs;
  duration_t compute_cost;
  ecn_ptr ecn;
  Rematerializer(const Unsafe&,
                 const rematerialize_function_t& func,
                 const strongs& inputs,
                 duration_t compute_cost)  :
    func(func),
    inputs(inputs),
    compute_cost(compute_cost) {
  }
  void release_resources() final {
    func = rematerialize_function_t();
    inputs.clear();
    outputs.clear();
  }
  void remat();
  ecn_ptr get_ecn();
  CheckpointInfo get_cpi();
};

ecn_ptr Rematerializer::get_ecn() {
  if (!ecn) {
    ecn = ecn_ptr::make(CheckpointInfo(compute_cost));
  }
  return ecn;
}

CheckpointInfo Rematerializer::get_cpi() {
  return CheckpointInfo(ecn ? duration_t(0) : compute_cost);
}

MakeRawResult make_raw(const rematerialize_function_t& remat_f,
```

```
37                          const strongs& inputs) {
38    for (const strong& s : inputs) {
39      s->pool->lock();
40    }
41    Tensors raw_inputs = uncheckpoint(inputs);
42    time_t pre = std::chrono::system_clock::now();
43    auto raw_outputs = remat_f(raw_inputs);
44    time_t post = std::chrono::system_clock::now();
45    pool.auto_evict();
46    base_compute_time_ += (post - pre).count();
47    std::vector<intrusive_ptr<External>> outputs;
48    std::vector<int> aliases;
49    weaks weak_outputs;
50    auto remat = intrusive_ptr<Rematerializer>::make(Unsafe(), remat_f, inputs,
    post - pre);
51
52    for (const Tensor& t : raw_outputs) {
53      intrusive_ptr<AliasPool> alias_pool;
54      int alias = get_alias(raw_inputs, t);
55      if (alias == -1) {
56        auto m = memory(t);
57        alias_pool = intrusive_ptr<AliasPool>::make(Unsafe(), remat, m);
58        pool.add(alias_pool);
59      }
60      else {
61        alias_pool = inputs[alias]->pool;
62        if (alias_pool->head_remat) {
63          alias_pool->head_remat->compute_cost += (post - pre);
64        }
65      }
66      auto e = intrusive_ptr<External>::make(t, alias_pool, remat);
67      pool.exts.push_back(weak_intrusive_ptr<External>(e));
68      alias_pool->tensors.push_back(weak(e->value));
69      outputs.push_back(e);
70      aliases.push_back(alias);
71      weak_outputs.push_back(weak(outputs.back()->value));
72    }
73    remat->outputs = weak_outputs;
74    for (size_t i = 0; i < inputs.size(); ++i) {
75      for (size_t j = 0; j < outputs.size(); ++j) {
76        if (!is_alias(raw_inputs[i], raw_outputs[j])) {
77          add_neighbor(inputs[i], outputs[j]->value);
78        }
79      }
80    }
81    for (const strong& s : inputs) {
82      s->pool->unlock();
```

```
  }
  return {outputs, aliases, post - pre, remat};
}
```

```
void AliasPool::set_not_evicted(const intrusive_ptr<AliasPool>& self) {
  if (unlikely(is_evicted)) {
    is_evicted = false;
    if (ecn) {
      TORCH_CHECK(head_remat);
      auto cpi = get_t(ecn);
      update_t(ecn, CheckpointInfo(cpi.compute_cost - head_remat->compute_cost));
      ecn.reset();
    }
    pool.add(self);
  }
}
```